

Lecture Summary: Program Execution, Interrupts & Bus Control

1. Program Execution Trace

The first part of the lecture focuses on **tracing the execution of a simple program** on a **hypothetical machine**, step by step.

Fetch–Execute Mechanism

Each instruction passes through the following stages:

1. The instruction located at the address in the **Program Counter (PC)** is fetched into the **Instruction Register (IR)**.
 2. The **PC is incremented** to point to the next instruction.
 3. The instruction is **decoded**.
 4. The instruction is **executed**.
-

Tracing Example (From the Lecture)

- The program starts at **memory address 300**.
 - Instructions used:
 - **LOAD**: Loads a value into the **Accumulator (AC)**.
 - **ADD**: Adds a value to the **AC**.
 - **STORE**: Stores the content of **AC** into memory.
-

I/O Instructions Introduced

- 0011: Load AC from an I/O device.
 - 0111: Store AC to an I/O device.
-

Registers Tracked During Execution

During tracing, the following registers are monitored after each step:

- PC
- IR

- AC
- MBR

This allows step-by-step observation of program execution.

2. Interrupts

An **interrupt** is an event that causes the CPU to **temporarily stop executing the current program** and switch to another task that requires immediate attention.

Types of Interrupts (*Very Important for Exam*)

1. External Interrupts

- Generated by **external I/O devices**.
 - Examples: keyboard input, disk I/O.
-

2. Internal Interrupts (Traps)

- Also called **Traps**.
- Caused by **errors inside the program**.
- Examples:
 - Division by zero
 - Invalid or illegal opcode

📌 **Highly repeated in exams**

3. Software Interrupts

- Generated by an **explicit instruction in the program**.
 - Often used to switch from **User Mode to Supervisor Mode**.
-

3. Interrupt Execution Cycle

When an interrupt occurs, the processor performs the following steps:

1. Completes execution of the **current instruction**.

2. Sends an **Interrupt Acknowledge (ACK)** signal to the interrupting device.
 3. Saves the current program state (especially **PC and registers**) in the **stack**.
 4. Loads the **PC** with the address of the **Interrupt Service Routine (ISR)**.
 5. Executes the ISR.
 6. Restores the saved registers and PC.
 7. Resumes execution of the original program.
-

4. Handling Multiple Interrupts

When multiple interrupts occur simultaneously, the CPU must decide **which interrupt to service first**.

Interrupt Arbitration Methods

1. Software Polling

- The CPU checks each I/O device sequentially.
 - Disadvantages:
 - Slow
 - Wastes CPU time
-

2. Daisy Chain (Hardware Polling) (*Very Important*)

- Devices are connected in a chain.
 - The **Interrupt Acknowledge** signal passes through devices in order.
 - The device that requested the interrupt captures the signal and sends its **interrupt vector**.
 - Priority depends on the device's position in the chain:
 - Closer to CPU → Higher priority
-

3. Multiple Interrupt Lines

- Each interrupt source has a separate line.
- The CPU selects the interrupt with the **highest priority**.

5. Bus Arbitration

Bus arbitration is required when **multiple devices request control of the system bus**.

Bus Arbitration Schemes

1. Centralized Arbitration

- A single **central arbiter** controls bus access.
 - The arbiter decides which device gets the bus.
-

2. Distributed Arbitration

- No central controller.
 - Each device contains its own arbitration logic.
 - Devices cooperate to determine bus ownership.
-

6. Important Exam Notes

- **Traps** are internal interrupts caused by errors such as division by zero.
- **Vector Interrupt:**
 - The interrupting device sends a **vector** to the CPU.
 - The vector identifies the device and the address of its ISR.
- **Daisy Chain priority** depends on the physical order of devices.